```
"""ViolationDashboard: the system-wide aggregation and
reporting layer."""

from collections import Counter

from .exceptions import PaymentError
from .payment import PaymentRecord


class ViolationDashboard:
    """Aggregates every ViolationRecord and PaymentRecord in
the system and
    produces the statistics and reports required by the state
transport
    authority.

    Design note: the dashboard *composes* ViolationRecords and
    PaymentRecords (it is the sole authoritative ledger; their
lifecycle
    is tied to it) but only *aggregates* references to Drivers,
Vehicles,
    EnforcementOfficers and AutomaticCameras (those objects are
created
    and owned independently and would continue to exist even if
the
    dashboard were discarded). See ``docs/design_notes.md``.
    """

    def __init__(self):
        self._violations: list = []  # composition: master
ledger
        self._payments: list = []  # composition: master ledger
        self._drivers: dict = {}  # aggregation: licence_no ->
Driver
        self._officers: dict = {}  # aggregation: badge_no ->
EnforcementOfficer
        self._cameras: dict = {}  # aggregation: camera_id ->
AutomaticCamera

    # ---- registration (aggregation -- dashboard does not own
lifecycle) --
    def register_driver(self, driver) -> None:
        self._drivers[driver.licence_no] = driver

    def register_officer(self, officer) -> None:
```

```
        self._officers[officer.badge_no] = officer

    def register_camera(self, camera) -> None:
        self._cameras[camera.camera_id] = camera

    # ---- ingest a violation issued by any Enforceable
(composition) ------
    def log_violation(self, violation) -> None:
        self._violations.append(violation)

    def record_payment(
        self, violation, amount_paid_ngn: float, channel,
payment_date=None
    ) -> PaymentRecord:
        """Validate and record full settlement of a violation's
fine
        (functional requirement #55). The full fine amount must
be paid
        in a single transaction; overpayment, underpayment, and
repeat
        payment of an already-settled violation are all
rejected."""
        if violation.paid:
            raise PaymentError(
                f"Violation {violation.violation_id} has
already been paid.",
                violation_id=violation.violation_id,
                amount_ngn=amount_paid_ngn,
            )
        if amount_paid_ngn != violation.fine_ngn:
            raise PaymentError(
                f"Amount NGN {amount_paid_ngn:,.2f} does not
match the "
                f"outstanding fine of NGN
{violation.fine_ngn:,.2f}.",
                violation_id=violation.violation_id,
                amount_ngn=amount_paid_ngn,
            )
        payment = PaymentRecord(violation, amount_paid_ngn,
channel, payment_date)
        violation.mark_paid()
        self._payments.append(payment)
        return payment

    # ---- analytics
```

```
----------------------------------------------------
    def total_fines_issued(self) -> float:
        return sum(v.fine_ngn for v in self._violations)

    def revenue_collected(self) -> float:
        return sum(p.amount_paid_ngn for p in self._payments)

    def outstanding_fines(self) -> float:
        return sum(v.fine_ngn for v in self._violations if not
v.paid)

    def top_offences(self, n: int = 5) -> list:
        counts = Counter(v.offence_code for v in
self._violations)
        return counts.most_common(n)

    def suspended_drivers(self) -> list:
        return [d for d in self._drivers.values() if
d.is_suspended]

    # ---- reporting
-----------------------------------------------------
    def dashboard_report(self) -> str:
        """Ranked top-5 offences, total revenue, outstanding
fines, and
        suspended driver count (functional requirement #56)."""
        lines = ["=" * 60, "VIOLATION DASHBOARD
REPORT".center(60), "=" * 60]
        lines.append("\nTop offences:")
        for code, count in self.top_offences(5):
            lines.append(f"  {code.ljust(24)}
{str(count).rjust(4)} violation(s)")
        lines.append(f"\nTotal revenue collected : NGN
{self.revenue_collected():>15,.2f}")
        lines.append(f"Total outstanding fines  : NGN
{self.outstanding_fines():>15,.2f}")
        lines.append(f"Suspended drivers        :
{len(self.suspended_drivers()):>15}")
        lines.append("=" * 60)
        return "\n".join(lines)

    def __len__(self) -> int:
        return len(self._violations)

    def __contains__(self, violation_id: str) -> bool:
```

```
        return any(v.violation_id == violation_id for v in
self._violations)

    def __str__(self) -> str:
        return f"ViolationDashboard({len(self._violations)}
violations logged)"

    def __repr__(self) -> str:
        return (

f"ViolationDashboard(violations={len(self._violations)}, "
            f"payments={len(self._payments)})"
        )
```